

Softwarearchitektur, Betrieb & Entwicklung

Lernzettel-App

Technische Dokumentation

Architektur, Service-Katalog, API, Setup, native Apps (Tauri 2) und Security der selbst-hostbaren, kollaborativen Lernplattform.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 2. Juli 2026

Inhaltsverzeichnis

1	Überblick	2
2	Architektur	3
2.1	Request-Fluss	3
2.2	Datenhaltung	4
3	Service-Katalog	5
3.1	Gateway-Routing	5
3.2	Gemeinsamer Code (<code>services/shared/</code>)	6
4	API & Authentifizierung	7
4.1	Login und Token-Verwendung	7
4.2	Öffentliche Endpunkte (ohne Token)	7
5	Setup & Betrieb	8
5.1	Installation	8
5.2	Deployment-Verifikation	8
5.3	OAuth einrichten (Google, GitHub, Apple)	9
5.3.1	Google	9
5.3.2	GitHub	9
5.3.3	Apple	9
6	Native Apps: Tauri 2	10
6.1	Besonderheiten gegenüber dem Browser	10
6.2	Desktop	10
6.3	Android	10
6.4	iOS / iPadOS	10
6.5	Bekannte Stolpersteine	11
7	Security	12
7.1	Behobene Probleme (Auswahl)	12
7.2	Akzeptierte Risiken	12
7.3	Key-Management	13
8	Entwicklung	14
8.1	Frontend	14
8.2	Einzelnen Service lokal starten	14
8.3	Konventionen	14
	Literaturverzeichnis	15

1 Überblick

Was ist die Lernzettel-App?

Eine **selbst-hostbare, kollaborative Lernplattform**: Markdown-/LaTeX-Editor mit Live-Vorschau, Quizzes mit server-seitiger Bewertung, Freunde & Sharing, PDF-Export über LuaLaTeX, Dokument-Vorlagen, Bibliographie (BibTeX/Zotero), AI-Tools (OpenAI, Anthropic, Ollama) und native Apps für Desktop, Android und iOS/iPadOS über Tauri 2 (Tauri Working Group [2026](#)).

Ebene	Technologie
Frontend	React 18, TypeScript, Vite; Theming über CSS-Variablen (Dark/Light/Dracula), i18n DE/EN
Backend	14 FastAPI-Microservices (Ramírez 2026) hinter einem API-Gateway
Persistenz	PostgreSQL 16, MongoDB 7, Datei-Volumes, Gitea (Versionierung)
Native Apps	Tauri 2 (eine Codebasis für Desktop, Android, iOS/iPadOS)
Deployment	Docker Compose, optional Cloudflare Tunnel

Legacy-Hinweis

Das Verzeichnis `backend/` im Repo ist ein ungenutztes Monolith-Relikt. Der aktive Backend-Code liegt vollständig unter `services/`.

2 Architektur

Die App ist eine Microservice-Anwendung: ein React-Frontend, ein API-Gateway und 13 fachliche Services, orchestriert über Docker Compose.

```
Browser ----> Frontend-nginx :3000 --/api--> Gateway :8000
Tauri-App -----(server_url)-----> Gateway :8000

Gateway :8000
|-- auth-service      :8001      (PostgreSQL)
|-- document-service  :8002      (PostgreSQL, uploads, Gitea)
|-- quiz-service      :8003      (PostgreSQL)
|-- latex-service     :8004      (Volume: PDFs)
|-- collab-service    :8005      (WebSocket-Relay, In-Memory)
|-- user-config-svc   :8006      (MongoDB)
|-- template-service  :8007      |-- image-service :8008
|-- github-service    :8009      |-- bib-service   :8010
|-- ai-service        :8011      |-- mcp-service   :8012
```

2.1 Request-Fluss

- 1 Browser:** lädt das Frontend vom nginx-Container (Port 3000); alle API-Aufrufe gehen relativ an `/api/...` und werden von nginx an das Gateway weitergereicht (inkl. WebSocket-Upgrade-Header).
- 2 Tauri-Apps:** haben keinen nginx-Proxy und sprechen das Gateway direkt an; die Basis-URL wird in den Einstellungen konfiguriert (localStorage `server_url`).
- 3 Gateway:** matcht den Pfad-Präfix (längster Präfix zuerst, mit Segment-Grenze — `/api/github` landet nicht bei `/api/git`) und leitet den Request mit einem geteilten `httpx.AsyncClient` weiter. Für `/api/collab/ws/{doc_id}` betreibt es ein bidirektionales WebSocket-Relay.
- 4 Services:** validieren das JWT selbst (shared `JWT_SECRET`, HS256) — es gibt keine implizite Vertrauensstellung zwischen den Services.

2.2 Datenhaltung

Speicher	Inhalt	Services
PostgreSQL	User, OAuth-Accounts, Dokumente, Shares, Kommentare, Freunde, Quizzes	auth, document, quiz
MongoDB	User-Konfiguration (Theme, Editor, PDF-Defaults), Profile	user-config
Gitea	Versionshistorie der Dokumente	document
Volumes	Uploads, generierte PDFs, Templates, Bilder, verschlüsselte User-Secrets	latex, template, image, github, bib, ai

Zero-Trust zwischen den Services

Jeder Service prüft eingehende JWTs eigenständig. Ein kompromittierter Service erhält dadurch keinen Freifahrtschein für die übrigen — und ein direkt (unter Umgehung des Gateways) angesprochener Service ist trotzdem authentifiziert.

3 Service-Katalog

Alle Backend-Services sind FastAPI-Apps (Python 3.12) mit eigenem `GET /api/health`. Nur Gateway, Frontend und Gitea publizieren Ports; alles andere ist ausschließlich über das Gateway erreichbar.

Service	Port	Aufgabe
gateway	8000	Reverse-Proxy, CORS, WebSocket-Relay, OpenAPI-Aggregation
auth-service	8001	Registrierung, Login, JWT, OAuth (Google/GitHub/Apple)
document-service	8002	Dokumente, Sharing, Kommentare, Freunde, Git-Historie
quiz-service	8003	Quizzes, Fragen, server-seitig bewertete Versuche
latex-service	8004	LuaLaTeX-Builds, Markdown→PDF, signierte Downloads
collaboration-service	8005	Live-Kollaboration über WebSockets (Rooms pro Dokument)
user-config-service	8006	User-Konfiguration, öffentliche Profile
template-service	8007	Dokument-Vorlagen mit Tags
image-service	8008	Bild-Uploads für Dokumente
github-service	8009	Push/Read von GitHub-Repos mit User-PATs
bibliography-service	8010	BibTeX-Verwaltung, Import/Export, Zotero-Sync
ai-service	8011	AI-Prompts/Generierung (OpenAI, Anthropic, Ollama)
mcp-service	8012	Model-Context-Protocol-Endpunkt (Tools, Resources, SSE)
frontend	3000	nginx: statisches React-Bundle + <code>/api</code> -Proxy
gitea	3001	Git-Server für die Dokument-Versionierung

Dazu kommen `db` (PostgreSQL), `mongo` (MongoDB) und optional `cloudflared` (Compose-Profil `tunnel`).

3.1 Gateway-Routing

Präfix	Ziel-Service
<code>/api/auth</code>	auth
<code>/api/documents</code>	document
<code>/api/comments</code>	document
<code>/api/friends</code>	document
<code>/api/git</code>	document
<code>/api/quiz</code>	quiz
<code>/api/latex</code>	latex
<code>/api/collab</code>	collaboration

Präfix	Ziel-Service
<code>/api/config</code>	user-config
<code>/api/profile</code>	user-config
<code>/api/templates</code>	template
<code>/api/images</code>	image
<code>/api/github</code>	github
<code>/api/bib</code>	bibliography
<code>/api/ai</code>	ai
<code>/api/mcp</code>	mcp

3.2 Gemeinsamer Code (`services/shared/`)

Modul	Zweck
<code>auth_utils.py</code>	JWT-Validierung; Start-Abbruch bei fehlendem/Default-Secret
<code>crypto_utils.py</code>	Fernet-Verschlüsselung für User-Secrets (Python Cryptographic Authority 2026)
<code>user_model.py</code>	dedupliziertes SQLAlchemy-User-Model

4 API & Authentifizierung

Alle Endpunkte laufen über das Gateway unter `http://<host>:8000/api/...`. Interaktive Dokumentation: `GET /api/docs` (Scalar-UI), Schema unter `GET /api/openapi.json`.

4.1 Login und Token-Verwendung

```
# 1. Token holen
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "alice", "password": "mein-passwort"}'
# -> {"access_token": "<JWT>", "token_type": "bearer", "user": {...}}

# 2. Token mitsenden
curl http://localhost:8000/api/documents \
  -H "Authorization: Bearer <JWT>"
```

JWT-Eckdaten

HS256, signiert mit dem shared `JWT_SECRET`; Laufzeit `JWT_EXP_MINUTES` (Default 1440 min = 24 h); Claims: `sub` (User-ID), `username`, `exp`.

Zwei Sonderfälle

WebSockets (`/api/collab/ws/{doc_id}`) akzeptieren keine Header — das Token wird als Query-Parameter `?token=<JWT>` übergeben. **PDF-Downloads** (`/api/latex/pdf/{id}?t=...`) nutzen ein kurzlebiges signiertes Download-Token aus der Build-Response, weil Browser-Downloads keine Authorization-Header senden.

4.2 Öffentliche Endpunkte (ohne Token)

- `POST /api/auth/register`, `POST /api/auth/login`, OAuth-Flow
- `GET /api/documents/shared/{token}` — öffentlich geteilte Dokumente
- `GET /api/profile/{user_id}` — öffentliches Profil
- `GET /api/health` pro Service

Alles andere erfordert ein gültiges Bearer-Token (sonst `401`). Fehlerformat ist FastAPI-Standard: `{"detail": "..."} mit passendem Statuscode (400/401/403/404, 429 beim Login-Rate-Limit, 502 bei Upstream-Fehlern).`

5 Setup & Betrieb

5.1 Installation

- 1 `.env` anlegen: `cp .env.example .env`
- 2 Pflichtwerte setzen (Compose bricht ohne sie ab), siehe Tabelle unten.
- 3 Stack starten: `docker compose up --build -d`
- 4 Prüfen: Frontend auf `:3000`, API-Docs auf `:8000/api/docs`, Gitea auf `:3001`.

Variable	Erzeugen mit
POSTGRES_PASSWORD	<code>openssl rand -hex 24</code>
JWT_SECRET	<code>openssl rand -hex 32</code>
SECRETS_ENCRYPTION_KEY	Fernet-Key, siehe Kommando unten

```
python3 -c "from cryptography.fernet import Fernet; \
print(Fernet.generate_key().decode())"
```

Key-Verlust ist endgültig

Geht der `SECRETS_ENCRYPTION_KEY` verloren, sind alle gespeicherten User-Secrets (AI-Keys, GitHub-PATs, Zotero-Keys) nicht mehr entschlüsselbar. Recovery: neuen Key setzen, alle User geben ihre Keys neu ein. Den Key wie ein Passwort sichern.

5.2 Deployment-Verifikation

```
docker compose ps # alles "running"/"healthy"?
curl -o /dev/null -w "%{http_code}\n" \
  http://localhost:8000/api/docs # -> 200

# Security-Regressionen:
# 6 Fehllogins -> 429 (Rate-Limit)
# /api/quiz ohne Token -> 401
# GET /api/ai/config -> nur "has_key", nie der Klartext-Key
# GET /api/documents/shared/<t> -> "share_token": null
```

Für die Live-Kollaboration zwei Browser-Fenster mit demselben Dokument öffnen — Änderungen müssen in beiden erscheinen (WebSocket-Relay über Gateway und nginx).

5.3 OAuth einrichten (Google, GitHub, Apple)

Provider ohne Konfiguration sind automatisch deaktiviert; das Frontend blendet die Buttons dynamisch ein (`GET /api/auth/oauth/providers`). Die Callback-URL ist immer

```
{OAUTH_REDIRECT_BASE}/api/auth/oauth/{provider}/callback.
```

5.3.1 Google

- 1 Google Cloud Console → „OAuth-Client-ID erstellen“ → Typ **Webanwendung**.
- 2 Weiterleitungs-URI: `.../api/auth/oauth/google/callback`.
- 3 `GOOGLE_CLIENT_ID` und `GOOGLE_CLIENT_SECRET` in die `.env`.

5.3.2 GitHub

- 1 GitHub → Settings → Developer settings → „New OAuth App“.
- 2 Callback-URL: `.../api/auth/oauth/github/callback`.
- 3 `GITHUB_CLIENT_ID` und `GITHUB_CLIENT_SECRET` in die `.env`.

5.3.3 Apple

Voraussetzungen

Benötigt einen **bezahlten Apple-Developer-Account** und eine **HTTPS-Callback-URL** — lokal daher nicht end-to-end testbar.

- 1 App ID mit „Sign in with Apple“ + zugehörige **Services ID** (= `APPLE_CLIENT_ID`) anlegen.
- 2 Return-URL bei der Services ID eintragen.
- 3 „Sign in with Apple“-Key erstellen, `.p8` herunterladen.
- 4 `APPLE_CLIENT_ID`, `APPLE_TEAM_ID`, `APPLE_KEY_ID`, `APPLE_PRIVATE_KEY` in die `.env` — das `client_secret` erzeugt der auth-service selbst als ES256-JWT.

Cookie-freier OAuth-Flow

Der Flow nutzt einen signierten State-JWT statt Session-Cookies und funktioniert dadurch identisch im Browser und aus den Tauri-Apps. Das Token wird im URL-Fragment übergeben (`#token=...`) und taucht damit in keinem Server-Log auf; native Apps empfangen es per Deep-Link `lernzettel://oauth/callback`.

6 Native Apps: Tauri 2

Die Apps in `tauri-app/` nutzen **dasselbe React-Frontend** aus `frontend/` — es gibt keine zweite UI-Codebasis.

6.1 Besonderheiten gegenüber dem Browser

Aspekt	Lösung
Server-Adresse	Kein <code>/api</code> -Proxy im Webview; Gateway-URL wird in Login/Einstellungen gesetzt (localStorage <code>server_url</code> , <code>/api</code> -Suffix wird ergänzt)
OAuth	Login öffnet den System-Browser; Rückweg per Deep-Link <code>lernzettel://oauth/callback#token=...</code>
Offline-Modus	Rust-Commands legen Dokumente im App-Data-Verzeichnis ab
CORS	<code>tauri://localhost</code> und <code>http://tauri.localhost</code> sind im Default von <code>ALLOWED_ORIGINS</code>

6.2 Desktop

```
cd tauri-app && npm install
npm run dev                # Vite (frontend/) + natives Fenster
npm run build              # Release-Bundle (dmg/msi/deb)
cd src-tauri && cargo check # schneller Rust-Check
```

6.3 Android

Voraussetzungen: Android Studio mit SDK + NDK, `ANDROID_HOME` gesetzt, Rust-Targets über `rustup` (eine Homebrew-Rust-Installation kann keine Cross-Targets nachladen).

```
rustup target add aarch64-linux-android armv7-linux-androideabi \
  i686-linux-android x86_64-linux-android
cd tauri-app
npx tauri android init    # erzeugt src-tauri/gen/android
npx tauri android dev     # Emulator/Geraet
npx tauri android build   # APK/AAB
```

Für den OAuth-Deep-Link im generierten `AndroidManifest.xml` einen Intent-Filter für das Schema `lernzettel` ergänzen (Details in `docs/mobile.md`).

6.4 iOS / iPadOS

Voraussetzungen: volles Xcode (nicht nur Command Line Tools), CocoaPods, Rust-Targets `aarch64-apple-ios(-sim)`.

```
cd tauri-app
npx tauri ios init      # erzeugt src-tauri/gen/apple
npx tauri ios dev       # iOS-Simulator
npx tauri ios build     # IPA (Signing noetig)
```

iPadOS

iPadOS ist dasselbe Target — in Xcode unter „Supported Destinations“ iPad aktivieren; das responsive Frontend deckt Phone- und Tablet-Breiten ab. Simulator-Betrieb geht ohne Bezahl-Account; echte Geräte, TestFlight und App Store brauchen einen Apple-Developer-Account.

6.5 Bekannte Stolpersteine

Problem	Ursache / Lösung			
<code>init</code> schlägt fehl	NDK bzw. Xcode fehlt; <code>npx tauri info</code> zeigt, was fehlt			
Weißes Fenster im Release	<code>frontend/dist</code> fehlt	—	vorher	
	<code>npm --prefix ../frontend run build</code>			
API nicht erreichbar	Server-Adresse nicht gesetzt; Android-Emulator: <code>10.0.2.2</code> statt <code>localhost</code>			
OAuth kommt nicht zurück	OAUTH_REDIRECT_BASE vom Gerät nicht erreichbar; Deep-Link-Schema prüfen			
Icons neu generieren	<code>npx tauri icon app-icon.png</code>			

7 Security

Zusammenfassung des Security-Audits (2026) mit den wichtigsten Findings und Fixes; Details in `docs/security.md` im App-Repo.

7.1 Behobene Probleme (Auswahl)

Schwere	Problem	Fix
Hoch	Quiz-Score spoofbar (Client sendete <code>correct</code> -Flags)	Server-seitiges Scoring; Antworten werden Nicht-Ownern nicht ausgeliefert
Hoch	Collaboration-WebSocket ohne Auth, <code>user_id</code> frei wählbar	JWT-Pflicht via <code>?token= ; user_id</code> aus dem Token
Hoch	LaTeX-Builds ohne Auth, shell-escape (RCE-Risiko)	Auth-Pflicht; <code>--safer / -no-shell-escape ;</code> Semaphore pro User; signierte Downloads
Hoch	<code>JWT_SECRET</code> mit Default-Wert	Start-Abbruch bei fehlendem/Default-Secret; Compose erzwingt den Wert
Hoch	User-Secrets im Klartext auf Disk, GETs echoten sie	Fernet-Verschlüsselung at rest; GET liefert nur <code>has_key</code>
Mittel	<code>share_token</code> -Leak an jeden Betrachter	Response strippt den Token
Mittel	Kein Login-Rate-Limit; Passwort-Minimum 4 Zeichen	5 Fehlversuche/15 min → 429; Minimum 10 Zeichen
Mittel	CORS * mit Credentials	Origins aus <code>ALLOWED_ORIGINS</code>
Mittel	Gateway: <code>/api/github</code> vom Präfix <code>/api/git</code> geschluckt	Longest-Prefix-Matching mit Segment-Grenze
Mittel	Collab im Docker-Deployment kaputt (kein WS-Proxying)	WebSocket-Relay im Gateway; nginx-Upgrade-Header

7.2 Akzeptierte Risiken

Risiko	Begründung
JWT im localStorage	Tauri-Apps brauchen direkten Token-Zugriff (Deep-Link-Login, WS-Query-Param); gemindert durch CSP und React-Escaping
Token im URL-Fragment	Fragmente erreichen keinen Server und keine Access-Logs; Callback-Seite entfernt das Fragment sofort
Shared <code>JWT_SECRET</code> (HS256)	Alle Services in derselben Compose-Trust-Boundary; bei Bedarf auf RS256 umstellbar
Gitea-Registrierung offen	Gitea ist standardmäßig nur lokal erreichbar; bei öffentlicher Exposition abschalten

7.3 Key-Management

Schlüssel	Hinweise
JWT_SECRET	Rotation loggt alle Nutzer aus, sonst folgenlos
SECRETS_ENCRYPTION_KEY	Bei Verlust sind verschlüsselte User-Secrets unwiederbringlich; sicher ablegen
POSTGRES_PASSWORD	Nur im Compose-Netz verwendet; DB publiziert keinen Host-Port
OAuth-Secrets	Liegen nur in der <code>.env</code> des Servers (gitignored)

Betriebsempfehlung

Öffentlich nur über HTTPS betreiben (z. B. Cloudflare Tunnel) — die JWTs sind Bearer-Tokens. `ALLOWED_ORIGINS` auf die tatsächlichen Domains einschränken und Basis-Images regelmäßig aktualisieren.

8 Entwicklung

8.1 Frontend

```
cd frontend && npm install
npm run dev          # http://localhost:5173 (Proxy /api -> :8000, ws:
                      true)
npx tsc --noEmit      # Type-Check
npm run build         # Produktions-Build nach dist/
npm run storybook     # Komponenten-Werkstatt auf :6006
```

8.2 Einzelnen Service lokal starten

```
cd services/auth-service
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export DATABASE_URL="postgresql+psycopg://lernzettel:<pw>@localhost:5432/
                      lernzettel"
export JWT_SECRET="dev-only" ALLOW_INSECURE_DEV=1 PYTHONPATH=..
uvicorn main:app --reload --port 8001
```

`ALLOW_INSECURE_DEV=1` niemals in Produktion setzen — es deaktiviert den Start-Abbruch bei fehlendem oder schwachem `JWT_SECRET`.

8.3 Konventionen

- Neue Endpunkte grundsätzlich mit `Depends(get_current_user)` absichern; JWT-Validierung über `services/shared/auth_utils.py`.
- User-Secrets nie im Klartext speichern (→ `services/shared/crypto_utils.py`).
- Neue API-Präfixe im Gateway (`SERVICE_ROUTES`) registrieren; das Frontend-nginx proxied nur `/api`.
- Frontend: Icons zentral aus `components/ui/icons.ts`; Farben, Abstände, Radii über die CSS-Tokens aus `themes.css`; schwere Abhängigkeiten nur in lazy geladenen Routen.
- Fehler dem User über die Toast-Komponente zeigen, keine `alert()`s.

Literaturverzeichnis

Python Cryptographic Authority (2026). *Fernet (symmetric encryption) — cryptography documentation*. URL: <https://cryptography.io/en/latest/fernet/> (besucht am 02.07.2026).

Ramírez, Sebastián (2026). *FastAPI — Framework Documentation*. URL: <https://fastapi.tiangolo.com> (besucht am 02.07.2026).

Tauri Working Group (2026). *Tauri 2.0 Documentation*. URL: <https://v2.tauri.app> (besucht am 02.07.2026).